

Machine Learning Methods for Finance

Supervised Learning - Introduction

Prof. Mike Aguilar

Economics Department and Fuqua School of Business, Duke University

<https://www.linkedin.com/in/mike-aguilar-econ>

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
Group similar assets by momentum characteristics
- **Dimension Reduction**
Condense multiple momentum measures into fewer factors
- **Generative Modeling**
Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
Predict positive/negative returns using momentum
- **Regression**
Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
Group assets by temporal behavior, then measure momentum
- **Supervised**
Forecast returns based on persistence

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Motivating with Momentum

- Suppose we want to predict returns with momentum
- Unlike unsupervised learning, we want to train our learning model on how to use momentum to make that prediction.
- The simplest supervised learner already in our toolbox is OLS;
$$r_i = \alpha + \beta Mom_i + u_i$$

More generally,

$$Y_i = \alpha + \beta X_i + u_i$$

- Estimate via OLS; denote $\hat{\theta} = \{\hat{\alpha}, \hat{\beta}\}$
- X may be vector ($N \times 1$) or a matrix ($N \times K$)
- X aka “features”; Y aka target
- If Y is continuous $\rightarrow$ regression. If dichotomous $\rightarrow$ classification
- Note: Regression and classification can be extended to a time series context, but we will eschew for now.

What is Supervised Learning?

Data: $(X_i, Y_i)_{i=1}^n$ drawn from joint distribution $P(X, Y)$.

Goal: Learn $f : \mathcal{X} \rightarrow \mathcal{Y}$ so predictions $\hat{Y} = f(X)$ are useful.

Population risk:

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[L(Y, f(X))]$$

Empirical risk:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n L(Y_i, f(X_i))$$

where L is some loss function and f is some functional form upon which we minimize empirical risk.

Functional Forms

Typical choices:

- Linear: $f(X) = \alpha + \beta^\top X$
- Trees, ensembles (RF, GBM)
- Neural nets
- k-NN, SVM

The more flexible the functional form, the more patterns it can fit.

Tradeoff: More flexibility can reduce bias, but increase variance.

Loss vs Evaluation Metric

- **Loss function (training):** A function $L(Y, f(X))$ minimized by the learning algorithm (often differentiable / convex). It drives parameter updates and defines the optimization objective.
- **Evaluation metric (reporting):** A performance measure used to compare models or to assess real-world usefulness. May be non-differentiable or domain-specific.
- **Key point:** *We optimize a loss; we evaluate with metrics.* They can differ (e.g., optimize log-loss but report AUC).
- **Practical implication:** Choose the loss to enable efficient learning and choose metrics that capture the business objective (e.g. calibration, ranking, cost-sensitive errors, etc...).

Common Loss Functions (used during training)

- **Regression**

$$L_{SE} = (Y - \hat{Y})^2 \quad (\text{squared error})$$

$$L_{LA} = |Y - \hat{Y}| \quad (\text{absolute error})$$

- **Classification / Probabilities**

$$L_{\log} = -(Y \log p + (1 - Y) \log(1 - p)) \quad (\log)$$

where p is the model's predicted probability that $Y = 1$

$$L_{\text{hinge}} = \max(0, 1 - y \cdot s(x)) \quad (\text{hinge})$$

- **Other useful training losses:** focal loss (imbalanced classes), quantile loss (conditional quantiles).

Bias–Variance Decomposition (Squared Loss)

For squared error prediction at a point x :

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

Interpretation:

- Complex models: $\downarrow$ bias, $\uparrow$ variance
- Simple models: $\uparrow$ bias, $\downarrow$ variance
- σ^2 is randomness in Y — even the true model cannot remove it

Goal: Select model complexity to balance bias and variance, then fit parameters to minimize training loss

Evaluation Metrics

Regression metrics

- RMSE: $\sqrt{\frac{1}{N} \sum (Y - \hat{Y})^2}$ — penalizes large errors
- MAE: $\frac{1}{N} \sum |Y - \hat{Y}|$ — robust to outliers
- R^2 (explained variance): $1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$

Classification metrics

- Accuracy, Precision, Recall, F1 (threshold-dependent)
- AUC / ROC (threshold-free ranking metric)
- Brier score (calibration of probabilistic forecasts)
- Log-loss (evaluates probabilistic accuracy)

- Residual vs fitted / feature plots: look for systematic patterns, which could signal misspecification
- QQ-plot of residuals: check distributional assumptions (if relevant)
- Heteroscedasticity check: residuals² vs features
- Confusion matrix & cost analysis (i.e. FP, FN, etc..) for operational decisions (use domain-specific costs)

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Why Not Evaluate on the Same 500 Assets?

Setting: 500 assets at time t . We use firm characteristics X_i to predict returns Y_i .

We estimate:

$$\hat{Y}_i = \hat{\alpha} + \hat{\beta}X_i \quad \text{using all 500 assets.}$$

Then we compute RMSE using those same 500 assets and take long positions in the assets with the highest expected returns and short positions in the lowest expected returns.

Q: Why is this not a reliable estimate of investment performance?

A:

- The model was chosen to fit these exact 500 observations
- If we allow enough flexibility, we can drive training error arbitrarily close to zero
- But that does not mean we can predict new data well

A Solution: Train / Test Splitting

Setting: 500 assets at time t . We use firm characteristics X_i to predict returns Y_i .

Step 1: Split the sample

- Training sample: randomly select 80% of assets (e.g., 400 firms)
- Test sample: use the remaining 20% of assets (e.g., 100 firms)

Step 2: Estimate model on training data

$$\hat{Y}_i^{Train} = \hat{\alpha}^{Train} + \hat{\beta}^{Train} X_i \quad \text{for } i \in \text{Training set}$$

Step 3: Evaluate on test data

$$\hat{Y}_j^{Test} = \hat{\alpha}^{Train} + \hat{\beta}^{Train} X_j \quad \text{for } j \in \text{Test set}$$

$$RMSE_{Test} = \sqrt{\frac{1}{N_{Test}} \sum_{j \in Test} (Y_j - \hat{Y}_j^{Test})^2}$$

Test performance estimates generalization error.

Train/Validation/Test

Many supervised models have **hyperparameters** τ :

- Ridge penalty λ
- Tree depth
- Number of boosting rounds
- Neural network architecture

These are not learned automatically — we must choose them. If we tune hyperparameters using the test set, we contaminate the test set.

Solution:

- Training set: estimate parameters θ
- Validation set: choose hyperparameters τ
- Test set: final unbiased evaluation

The Problem With Single Splits

Q: What's the problem with use a single train, single validate, single test?

A single train/validation/test split depends on:

- Which observations were held out, and
- ...random sampling variation, which can
- ...cause the test error estimate can have high variance.

Solution: Cross-validation, which reduces variance by averaging across multiple splits.

K-Fold Cross-Validation

Repeatedly hold out data

Example: 500 assets in training set. Choose $K = 5$

- Split data into 5 equal groups (“folds”) of 100 assets each.
- A **fold** is simply one subset used as temporary validation data.

For $k = 1, \dots, 5$:

- Train the model on 4 folds (400 assets)
- Compute validation error on the held-out fold (100 assets)
- Record validation error R_k

Cross-validated error:

$$\hat{R}_{CV} = \frac{1}{K} \sum_{k=1}^K R_k$$

Use $\hat{R}_{CV}$ to compare models or tune hyperparameters.

How Do We Choose K ?

Choosing K controls the bias–variance tradeoff of the cross-validation error estimate. Q: If K is small, is bias higher/lower? Is variance higher/lower?

Small K (e.g., 2–3)

- Larger validation sets
- Higher bias (model trained on less data)
- Lower variance

Large K (e.g., Setting $K = N$ is referred to LOOCV, Leave One Out CV)

- Larger training sets
- Low bias
- High variance
- Computationally expensive

Hyperparameter Tuning with CV

Step 1: Split data into Training and Test

Step 2: Use K-fold cross-validation on the Training set, where each fold temporarily acts as a validation set.

Algorithm:

- 1 Define hyperparameter grid $\{\tau\}$
- 2 For each τ , compute cross-validated risk $\hat{R}_{CV}(\tau)$
- 3 Select $\tau^* = \arg \min \hat{R}_{CV}(\tau)$

Step 3: Refit model on full Training set using τ^*

Step 4: Evaluate once on Test set

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Feature Importance: Linear vs Nonlinear

Linear models: coefficient magnitude/sign

Nonlinear models:

- Global: permutation importance, mean absolute SHAP
- Local: SHAP values, LIME
- Functional: Partial Dependence Plots (PDP), ICE plots

SHAP: Concept and Mathematics

Goal: quantify each feature's contribution to an individual prediction.

Shapley value formula:

$$\phi_j(x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(x_{S \cup \{j\}}) - f(x_S)] .$$

- F = features, S subset without j .
- $f(x_S)$: model prediction using features in S (others marginalized).
- Additive decomposition: $f(x) = \phi_0 + \sum_{j=1}^p \phi_j(x)$
- $\phi_j(x)$: the contribution of feature j to the model's prediction for the observation with feature vector x , relative to a baseline ϕ_0

SHAP Example: Explaining Cross-Sectional Fit

Setting: 500 assets at a single date t . We model cross-sectional returns as $r_i = f(X_i)$, where X_i includes: momentum, size, value, volatility

For Asset A, the model predicts:

$$\hat{r}_A = 2.5\%$$

Average fitted return Shapley value across all 500 assets:

$$\phi_0 = 1.0\%$$

SHAP decomposition for Asset A:

$$2.5\% = 1.0\% + 0.9\%_{\text{Momentum}} - 0.2\%_{\text{Size}} + 0.6\%_{\text{Value}} + 0.2\%_{\text{Volatility}}$$

Interpretation:

- Momentum, value, and to a lesser extent volatility, push this asset above the cross-sectional average.
- Large size slightly lowers its fitted return.
- Contributions sum exactly to the fitted value.

Partial Dependence Plots (PDP)

Goal: Visualize average marginal effect of feature x_j .

$$PD_j(z) = \frac{1}{n} \sum_{i=1}^n f(z, x_{i,-j})$$

- z grid for x_j , $x_{i,-j}$ other features for obs i .
- PDP displays expected prediction as x_j varies (averages out other features).
- Caveat: PDP can obscure heterogeneity if interactions are strong; consider ICE plots.

Partial Dependence Plots (PDP)

Goal: How does the model's fitted return change as one feature varies?

Setting: 500 assets at a single date. We model cross-sectional returns
 $\hat{r}_i = f(X_i)$

For feature x_j (e.g., Momentum):

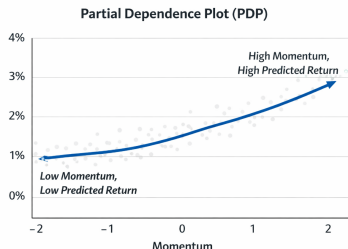
$$PD_j(z) = \frac{1}{n} \sum_{i=1}^n f(z, x_{i,-j})$$

Meta Algorithm:

- Fix Momentum at value z for all assets
- Keep all other characteristics at their observed values
- Average the model's predicted returns across all assets
- Repeat for a grid of z values
- Create the PDP curve that traces the model-implied average relationship between Momentum (z) and average predicted returns

Partial Dependence Plot

Setting: 500 assets at a single date. We model cross-sectional returns $\hat{r}_i = f(X_i)$



- The curve shows the model-implied average relationship between Momentum and fitted return.
- Higher momentum is associated with higher fitted returns.
- This reflects how the *model* uses Momentum, not necessarily a causal effect.

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Many Features

Q: Advantage of many features?

A: Lower bias / better predictions

Q: Disadvantage of many features?

A:

- Large K expands hypothesis space and variance
- Linear models: $\hat{\beta} = (X'X)^{-1}X'y$ becomes unstable when $X'X$ ill-conditioned
- Nonlinear models: more candidate splits/parameters $\rightarrow$ more opportunity to fit noise

How to Handle Many Features

Main strategies:

- 1 Dimension reduction (PCA, autoencoders, supervised screening)
- 2 Regularization (Ridge, Lasso, Elastic Net)
- 3 Model averaging
- 4 Ensembles (Bagging, Boosting, Stacking)
- 5 Super Learner (CV-based optimal ensemble)

Regularization (Ridge / Lasso / Elastic Net)

Ridge (L2):

$$\hat{\beta}^{ridge} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$$

- Shrinks coefficients toward zero.
- Rarely sets coefficients exactly to zero.
- Stabilizes estimates under multicollinearity.

Lasso (L1):

$$\hat{\beta}^{lasso} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

- Shrinks and can set coefficients exactly to zero.
- Performs automatic variable selection.
- Can be unstable when features are highly correlated.

Regularization (Ridge / Lasso / Elastic Net)

Elastic Net:

$$\hat{\beta}^{EN} = \arg \min_{\beta} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda(\alpha \|\beta\|_1 + (1 - \alpha) \|\beta\|_2^2)$$

Combines shrinkage and variable selection; useful when predictors are correlated.

Model Averaging and Bagging

Model Averaging: Combine predictions from multiple models:

$$\hat{y}(x) = \sum_{m=1}^M w_m \hat{f}_m(x)$$

Averaging reduces variance when models make different errors.

Bagging (Bootstrap Aggregating):

- Draw B bootstrap samples from the training data.
- A **bootstrap sample** is drawn with replacement.
- Fit a model on each sample: $\hat{f}_b(x)$.
- Average predictions:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

Bagging primarily reduces variance. e.g. Random Forest

Boosting and Stacking

Boosting: Sequentially fit weak learners to residuals:

$$f_m(x) = f_{m-1}(x) + \nu h_m(x)$$

Each learner focuses on correcting previous mistakes.

- $h_m(x)$: the m -th weak learner fit to current residuals
- ν : learning rate (shrinks each learner's contribution)

Examples: AdaBoost, Gradient Boosting, XGBoost.

Stacking:

Train multiple base learners and combine via a meta-learner:

$$\hat{y}(x) = g(\hat{f}_1(x), \dots, \hat{f}_M(x))$$

- CV-based ensemble that learns weights to minimize estimated risk.

$$\min_{\alpha} \sum_{m=1}^M \alpha_m \hat{R}_m \quad \text{s.t.} \quad \alpha_m \geq 0, \quad \sum_m \alpha_m = 1$$

- Asymptotically performs as well as the best candidate in the library.

Outline

- 1 Foundations: Problem, Loss, Risk
- 2 Train / Validate / Test & Cross-Validation
- 3 Feature Importance & Interpretability
- 4 Many Features, Remedies
- 5 Tips and Workflow

Data Preprocessing aka Feature Engineering

- Standardize/normalize features (learn scaling on training only).
- Encode categorical variables (one-hot, target encoding — with care).
- Impute or model missingness (use training folds for imputation).
- Think carefully about interactions
- Be sure to apply transformations consistently across features

Data Leakage (Pitfalls)

Data leakage = test information used during training/tuning.

Examples:

- Scaling using full dataset before split.
- Feature selection (or imputation) that used test labels.
- Using future information for features in a forecasting context.

Result: over-optimistic performance and models that fail in production.

Recommended Practical Workflow

- 1 Define problem, loss, and metric.
- 2 Split data: train / validate (CV) / test.
- 3 Preprocess using training folds only.
- 4 Baseline with simple model (OLS, logistic).
- 5 Perform feature engineering and selection inside CV.
- 6 Tune hyperparameters via CV.
- 7 Evaluate on held-out test set.
- 8 Inspect interpretability (SHAP, PDP), calibration, and robustness (stress tests).